

splash view

关键字：splash 性能 骨架屏 loading splashView

当一个页面加载时间较长时，为优化体验，通常先展示部分信息或页面骨架，此功能提供一个加载预展示页面的通用解决方案

一、使用说明

1、SplashView是 [【配置文件】](#) 中配置项的一部分，若使用SplashView需先添加当前页面的 [【配置文件】](#)。

2、配置文件中添加 splash 字段，属性说明如下

- xmlUrl {string} 可选，splash view 展示内容的文件名（文件名必须在当前项目保持唯一，不唯一时刻使用相对路径）
- cssUrl {string} 可选，splash view 展示样式的文件名（文件名必须在当前项目保持唯一，不唯一时刻使用相对路径）
- data {JSON string} 可选，splash view 展示内容对应的数据，只能有一层数据，即 value 必须是基本类型 * 注意，**千万要注意data是JSON字符串类型，需要用JSON.stringify处理一下**。如果在native中传值，需要转换为AJX的单位 * 例如:iOS(point/BS_POINT_SCALE)

示例：

∨ 复制代码

```
{
  "showMap":true, //可选，true表示当前ajax页面需要显示地图，false时不需要显示地图。
  "locationMode":1, //可选，不写或0表示默认，1=无定位，2=GPS，3=网络，4=GPS+网络，注意是数字！不要写字符串
  "splash": { //SplashView的配置信息
    "xmlUrl": "xxxpage.xml",
    "cssUrl": "xxxpage.css",
    "data": JSON.stringify({
      "somedata": "1",
      "somedata2": "str"
    })
  }
}
```

3、打开 ajax 页面时，客户端如果检测到配置了 SplashView 的参数，则启动 SplashView 逻辑

- 4、SplashView 大小不支持配置，与对应的页面大小保持一致，并处于完全相同的位置，以达到覆盖页面的目的
- 5、SplashView 仅支持解析静态的 xml css 来渲染，无 js 运行环境，以达到最快的展示速度
- 6、**SplashView 不会在页面加载后自动关闭，需要对应页面在合适的时机主动调用关闭方法 `ajax.ui.dismissSplashView()`**
- 7、页面配置SplashView后，不论以何种方式（navigator.openxxx、scheme 跳转）打开此页面，都会展示SplashView

二、可通过打开页面接口时传参来覆盖配置文件中 SplashView

当页面配置SplashView并且通过`ajax.navigator.openPath`或`ajax.navigator.openScheme`打开此页面时传入了splash配置时，传入的配置覆盖对应的默认配置。`ajax.navigator.openPath`或`ajax.navigator.openScheme`详细使用方式参见 [【navigator】](#)

splash 属性说明

- `xmlTag {string}` 可选，splash view 展示内容的协议名，在目标页面的项目 bundle.json 中配置 [协议文档](#)
- `cssTag {string}` 可选，splash view 展示样式的协议名，在目标页面的项目 bundle.json 中配置 [协议文档](#)
- `xmlUrl {string}` 可选，splash view 展示内容的文件路径，通过 `require.toUrl` 获取
- `cssUrl {string}` 可选，splash view 展示样式的文件路径，通过 `require.toUrl` 获取
- `data {JSON string}` 可选，splash view 展示内容对应的数据，只能有一层数据，即 value 必须是基本类型 * 注意，**千万要注意data是JSON字符串类型，需要用JSON.stringify处理一下**，如果在native中传值，需要转换为AJX的单位 * 例如:iOS(point/BS_POINT_SCALE)

注意：

- **xmlTag 和 xmlUrl 只能二选一**
- `cssTag` 必须和 `xmlTag` 配合使用，`cssTag` 可缺省
- `cssUrl` 必须和 `xmlUrl` 配合使用，`cssUrl` 可缺省
- `xmlTag` 和 `cssTag` 新增于 10.13.0 版本
- 如果打开的页面与当前页面不在同一 bundle，不能使用 `xmlUrl` 方式，可能导致读取 `xmlUrl` 对应的文件失败，应该配置 `xmlTag` 跳转的目标页面所在的 bundle 内

navigator.openPath 示例

示例：



复制代码

```
// xmlUrl (splash的xml和css资源在当前bundle内)
const options = {
  pageUrl: 'path://ajxdemo/xxxpage.page.js',
  pageId: 'NonFoxPageB',
  param: {
    username: 'handy.wang', //业务数据
    password: '111111' //业务数据
  },
  splash: {
    xmlUrl: require.toUrl('NonFoxPageSplash.xml'),
    cssUrl: require.toUrl('NonFoxPageSplash.css'),
    data: JSON.stringify({
      somedata: '1',
      somedata2: 'str',
    }),
  },
};
ajx.navigator.openPath(options);
```

```
// xmlTag (splash的xml和css资源在跨bundle内)
const options2 = {
  pageUrl: 'path://ajxdemo/xxxpage.page.js',
  pageId: 'NonFoxPageB',
  param: {
    username: 'handy.wang', //业务数据
    password: '111111' //业务数据
  },
  splash: {
    xmlTag: 'spalsh/poi/hotel_xml',
    cssTag: 'spalsh/poi/hotel_css',
    data: JSON.stringify({
      somedata: '1',
      somedata2: 'str',
    }),
  },
};
ajx.navigator.openPath(options2);
```

navigator.openScheme 示例

参数 key 为 splash，值为 json 字符串，json 内容路径需要为绝对路径，其余与默认配置相同

示例：

复制代码

```
// 此处用 js 语法拼装，native 跳转也相同
// js 拼接 url 时，如果文件为当前 bundle 内文件，可使用 require.toUrl 来转换
// 路径

// xmlUrl (splash的xml和css资源在当前bundle内)
const config = {
  xmlUrl: 'path://xxbundle/xxdir/xxfile.xml',
  cssUrl: 'path://xxbundle/xxdir/xxfile.css',
  data: JSON.stringify({
    somedata: '1',
    somedata2: 'str'
  })
};
const scheme = `amapuri://xxxpage?
splash=${encodeURIComponent(JSON.stringify(config))}`
ajax.navigator.openScheme(scheme);

// xmlTag (splash的xml和css资源在跨bundle内)
const config = {
  xmlTag: 'spalsh/poi/hotel_xml',
  cssTag: 'spalsh/poi/hotel_css',
  data: JSON.stringify({
    somedata: '1',
    somedata2: 'str'
  })
};
const scheme = `amapuri://xxxpage?
splash=${encodeURIComponent(JSON.stringify(config))}`
ajax.navigator.openScheme(scheme);
```

三、SplashView模板语法

xml 中可以使用双方括号 `[[[]]]` 来包裹一个变量，展示时会被替换为 data 中的对应属性的值

如果变量在 data 中未找到，则替换为空字符串

示例：

复制代码

```
<div class="splash">
  <label text="[[name]]" />
  <label text="[[address]]" />
  <div style="display: [[block1Display]];"></div>
  <div style="display: [[block2Display]];"></div>
</div>
```

data数据:



复制代码

```
JSON.stringify({
  "name": "首开",
  "block1Display": "flex",
  "block2Display": "none"
})
```

最终展示 xml



复制代码

```
<div class="splash">
  <label text="首开" />
  <label text="" />
  <div style="display: flex;"></div>
  <div style="display: none;"></div>
</div>
```